

SH2150 Homework 3

Gustav Marthinsson, Scott McHaffie

December 11, 2025

Diffusion models have become one of the most exciting ideas in modern generative AI, and at their core is a beautiful interplay between randomness and structure described by stochastic differential equations (SDEs). In this assignment, we dive into that world: from exploring classical processes like Brownian motion and the Ornstein–Uhlenbeck dynamics, to simulating trajectories with Euler–Maruyama, to building neural networks that learn how data is gradually corrupted and then reconstructed. By moving from simple 1D SDEs, all the way to UNet-based image models, we get to experience how mathematical theory, numerical simulation, and deep learning come together to form the foundation of today’s powerful diffusion and flow-based generative models.

Your tasks are to

- Get familiar with SDEs
- Set up a training/inference pipeline for generative models
- Sample from a 2D distribution using a flow model
- Define a UNet architecture for image generation
- Generate images from a distribution of your choice

1. Theoretical background

Stochastic differential equations (SDEs) are given on the form:

$$dX_t = u_t(X_t)dt + \sigma_t dW_t, \quad X_0 = x_0 \quad (1)$$

Where X_t is the trajectory, u_t is the drift coefficient and σ_t is the diffusion coefficient. W_t denotes the *Wiener process*, or Brownian motion. This term gives rise to the stochastic properties of the system.

(a) Different choices of u_t yield different stochastic processes that can be used to model many different physical phenomena. The Ornstein–Uhlenbeck (OU) process is given by $u_t(X_t) = -\theta X_t$, $\sigma_t = \sigma$, for some constants θ, σ . It can be used to model massive Brownian motion with a dependence

of friction. Show that the OU process is equivalent to Langevin dynamics defined by the drift coefficient

$$u_t(X_t) = \frac{1}{2}\sigma_t^2 \nabla \log p(X_t), \quad \sigma_t = \sigma, \quad (2)$$

when $p(X_t) = \mathcal{N}(0, \frac{\sigma^2}{2\theta})$.

Answer: We consider the Ornstein-Uhlenbeck (OU) process for an SDE which is given by $u_t(X_t) = -\theta X_t$, $\sigma_t = \sigma$ for some constants θ, σ . We want to show that the OU-process is equivalent to Langevin dynamics defined by:

$$u_t(X_t) = \frac{1}{2}\sigma_t^2 \nabla \log p(X_t), \quad \sigma_t = \sigma, \quad p(X_t) = \mathcal{N}\left(0, \frac{\sigma^2}{2\theta}\right) \quad (3)$$

We have our SDEs for each respective case as:

$$\text{OU : } dX_t = -\theta X_t dt + \sigma dW_t \quad (4)$$

$$\text{Langevin : } dX_t = \frac{1}{2}\sigma^2 \nabla \log p(X_t) dt + \sigma dW_t \quad (5)$$

For these to be equivalent, the drift term must be equal for both cases; we require $-\theta X_t = \frac{1}{2}\sigma^2 \nabla \log p(X_t)$. We start by considering the probability density function for an arbitrary Gaussian $\sim \mathcal{N}(\mu, \nu^2)$:

$$p(x) = \frac{1}{\sqrt{2\pi\nu^2}} \exp\left(-\frac{(x-\mu)^2}{2\nu^2}\right) \quad (6)$$

In our case, we have $\mu = 0, \nu^2 = \sigma^2/2\theta$:

$$\implies p(x) = \sqrt{\frac{2\theta}{2\pi\sigma^2}} \exp\left(-\frac{2\theta x^2}{2\sigma^2}\right) = \sqrt{\frac{\theta}{\pi}} \frac{1}{\sigma} \exp\left(-\frac{\theta x^2}{\sigma^2}\right) \quad (7)$$

Now, we compute the logarithm of this:

$$\log \left(\sqrt{\frac{\theta}{\pi}} \frac{1}{\sigma} \exp \left(-\frac{\theta x^2}{\sigma^2} \right) \right) = \log \left(\sqrt{\frac{\theta}{\pi}} \frac{1}{\sigma} \right) + \log \left(\exp \left(-\frac{\theta x^2}{\sigma^2} \right) \right) \quad (8)$$

$$\implies \log(p(x)) = \log \left(\sqrt{\frac{\theta}{\pi}} \frac{1}{\sigma} \right) - \frac{\theta x^2}{\sigma^2} \quad (9)$$

$$\frac{d}{dx} (\log[p(x)]) = -\frac{2\theta x}{\sigma^2} \quad (10)$$

Inserting this into the SDE for the Langevin dynamics, with $x = X_t$ yields:

$$dX_t = \frac{1}{2}\sigma^2 \left(-\frac{2\theta X_t}{\sigma^2} \right) dt + \sigma dW_t \quad (11)$$

$$\implies dX_t = -\theta X_t dt + \sigma dW_t \quad (12)$$

This is exactly the SDE for the Ornstein-Uhlenbeck process.

(b) Being able to simulate SDEs will be needed to sample new data points from generative models. The simplest solver for SDEs is the Euler–Maruyama method defined as

$$X_{t+h} = X_t + h u_t(X_t) + \sqrt{h} \sigma_t \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I), \quad (13)$$

where h is the step size. This reduces to the Euler method for ODEs when $\sigma = 0$. Implement the Euler–Maruyama method.

Answer: We implemented the Euler–Maruyama in the `.ipynb` notebook.

(c) Simulate a Brownian motion defined by $\sigma_t = \sigma$, $u_t = 0$, $X_0 = 0$. What happens for different values of σ ?

Answer: We simulated Brownian motion for different values of σ , namely $\sigma = 0.01, 0.1, 1.0, 10.0$ and the results are shown in Figure 1 to Figure 4.

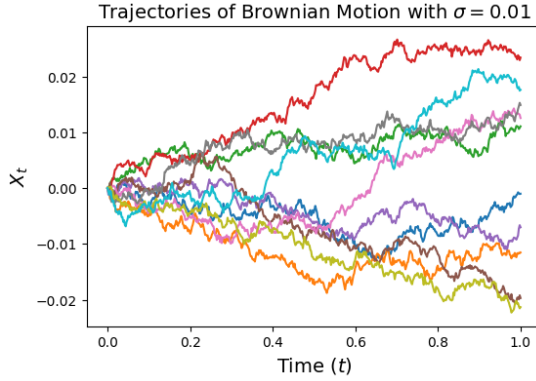


Figure 1: Brownian motion for $\sigma = 0.01$

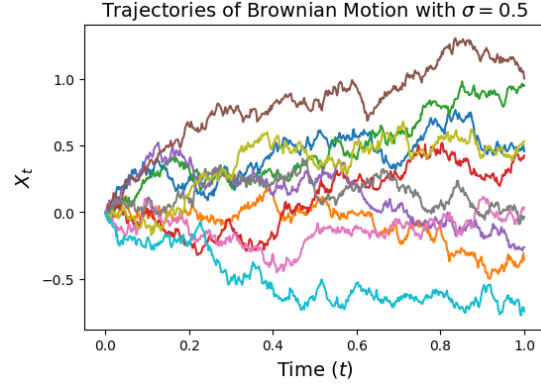


Figure 2: Brownian motion for $\sigma = 0.5$

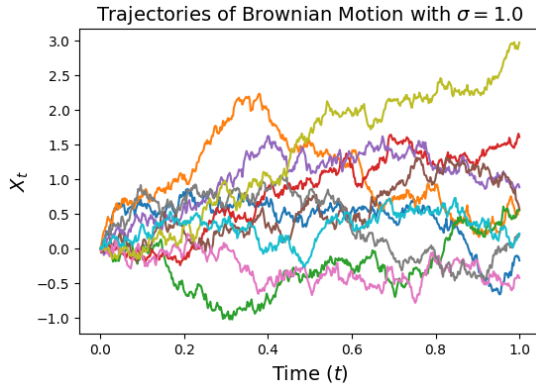


Figure 3: Brownian motion for $\sigma = 1$

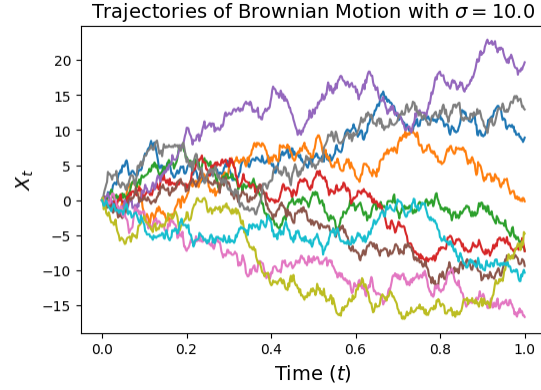


Figure 4: Brownian motion for $\sigma = 10$

For the different values of $\sigma \in \{0.01, 0.5, 1, 10\}$ it can clearly be seen that $|X_t|$ is σ -dependent with low values of $|X_t|$ for low σ and high $|X_t|$ for high σ . This is expected for Brownian motion, as the Euler-Maruyama relation in eq (12) reduces to:

$$X_{t+h} = X_t + \sqrt{h}\sigma\epsilon_t \quad (14)$$

The scaling of the Gaussian noise term is proportional to σ , so for high σ we expect a more noisy trajectory with a more deviating path and the contrary for low values.

(d) Simulate the OU process defined above. Run it with a variety of different initial points X_0 . Pay attention to the ratio $\frac{\sigma^2}{2\theta}$ and compare the long-time behavior of the solutions. Are they approaching a particular point or a distribution?

Answer: We implement the OU-process defined in eq(3) for initial values in the range $X_0 \in [-10, 10]$ with Euler-Maruyama and different values of θ, σ .

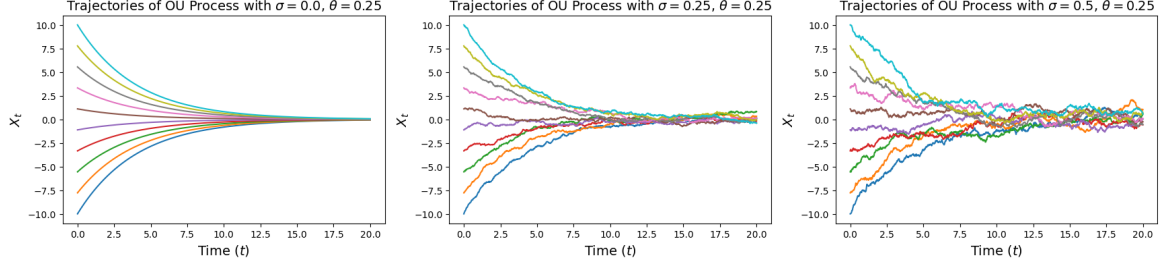


Figure 5: OU trajectories for $\theta = 0.25$ and $\sigma \in \{0, 0.25, 0.5\}$

In the case where $\sigma = 0$, the stochastic term vanishes and the OU process reduces to a purely deterministic system:

$$dX_t = -\theta X_t dt \quad (15)$$

The solution to this equation is simply an exponential function:

$$X_t = X_0 e^{-\theta t} \quad (16)$$

As the diffusion term is increased, more stochastic behavior is introduced into the system; we recall that the Gaussian noise strength in Euler-Maruyama is proportional to σ . Solutions now include the stochastic term:

$$X_t = X_0 e^{-\theta t} + \sigma \int_0^t e^{-\theta(t-s)} dW_s \quad (17)$$

We investigate the convergence of Equation 16; the deterministic term $X_0 e^{-\theta t} \rightarrow 0$ as time propagates. Since the stochastic term is Brownian it has mean 0, so $E[X_t] = E\left[\sigma \int_0^t e^{-\theta(t-s)} dW_s\right] = 0$ for some t such that the deterministic term is small enough to neglect. Furthermore, Brownian motion is Gaussian with some variance ν^2 so we know that $X_t \sim \mathcal{N}(0, \nu^2)$. The variance can be found as $Var(X_t) = E[X_t^2] - E[X_t]^2 = E[X_t^2]$, since $E[X_t] = 0$. Through Itô isometry we have:

$$E\left[\left(\int_0^T X_t dW_t\right)^2\right] = E\left[\int_0^T X_t^2 dt\right] \quad (18)$$

For some Stochastic process governed by Brownian motion. This corresponds exactly to the term in our variance equation; computing it with the integral from the OU process yields:

$$Var(X_t) = E[X_t^2] = \int_0^t \left(\sigma e^{-\theta(t-s)}\right)^2 ds = \sigma^2 \int_0^t e^{-2\theta(t-s)} ds = \frac{\sigma^2}{2\theta} (1 - e^{-2\theta t}) \quad (19)$$

Again, we assume t to be large enough at convergence so that the exponential term goes to 0. We are then left with a variance $Var(X_t) = \frac{\sigma^2}{2\theta}$ which we recognize from the first assignment, and it can be concluded that the solutions converge towards the distribution:

$$X_t \sim \mathcal{N}\left(0, \frac{\sigma^2}{2\theta}\right), \quad t \rightarrow \infty \quad (20)$$

For the case where $\sigma = 0$, the distribution collapses into a point convergence $X_t = 0, \sigma = 0, t \rightarrow \infty$. The convergence rate is dependent on θ , which can be seen from eq. (15); larger θ means higher convergence rate. We plotted the results from Figure 5 again but this time with a value of $\theta = 0.5$. These results are shown in Figure 6 and it can be seen that the convergence occurs faster now.

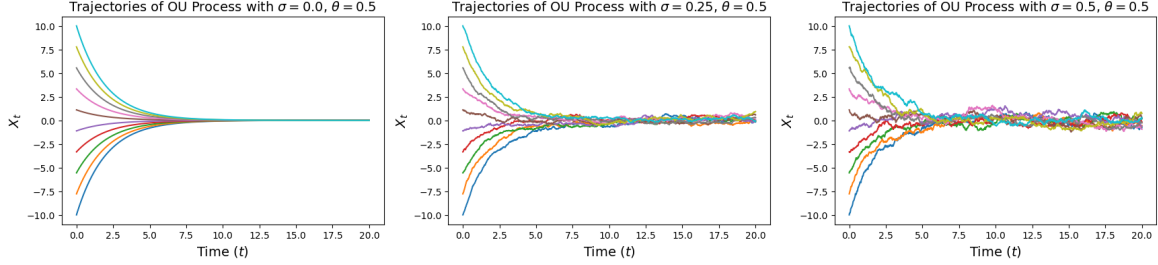


Figure 6: OU trajectories for $\theta = 0.5$ and $\sigma \in \{0, 0.25, 0.5\}$. Note the faster convergence

2. A diffusion model is an SDE whose vector field is parameterized by a learnable neural network u_θ^t .

In a diffusion model, the vector field u_t is parametrized by a neural network u_t^θ that is trained on data points $X_1 \sim p_{data}$. These data points have been corrupted with varying amounts of noise for different time steps $X_t, t \in [0, 1]$.

For a denoising diffusion model, the conditional path is given by:

$$x_t = \alpha_t z + \beta_t \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (21)$$

α_t, β_t are continuously differentiable and monotonic noise schedulers with $\alpha_0 = \beta_1 = 0$ and $\alpha_1 = \beta_0 = 1 \implies x_t \sim \mathcal{N}(\alpha_t z, \beta_t^2 I)$.

(a) Implement linear noise schedulers $\alpha_t = t, \beta_t = 1 - t$.

We implemented the linear noise schedulers as

$$\alpha_t = t \quad (22)$$

$$\beta_t = 1 - t, \quad (23)$$

in our `.ipynb` notebook.

(b) Implement the conditional sampler that given a data point z and a time t returns the corrupted data point

$$z_t = \alpha_t z + \beta_t \epsilon.$$

Answer: We implemented the conditional sampler in our `.ipynb` notebook.

(c) Define a 2D toy dataset (e.g. a Gaussian mixture) p_{data} . Simulate the corruption process on 1000 samples from p_{data} for $t = 0, 0.25, 0.50, 0.75, 1$, and plot the 2D histograms.

Answer: We defined a noise distribution using our `IsotropicGaussian` class, generating a 2D noise distribution with a standard deviation of $\sigma = 1$. We also defined a 2D toy dataset of 5 Gaussian blobs using our `GaussianMixture.symmetric_2D` class. We defined the mixture of 5 Gaussians to have a standard deviation of 0.5, a scale of 5.0, and an x offset of 0.0. The noise distribution, the Gaussian mixture, and the two datasets are shown in Figure 7 on the left, center, and right, respectively.

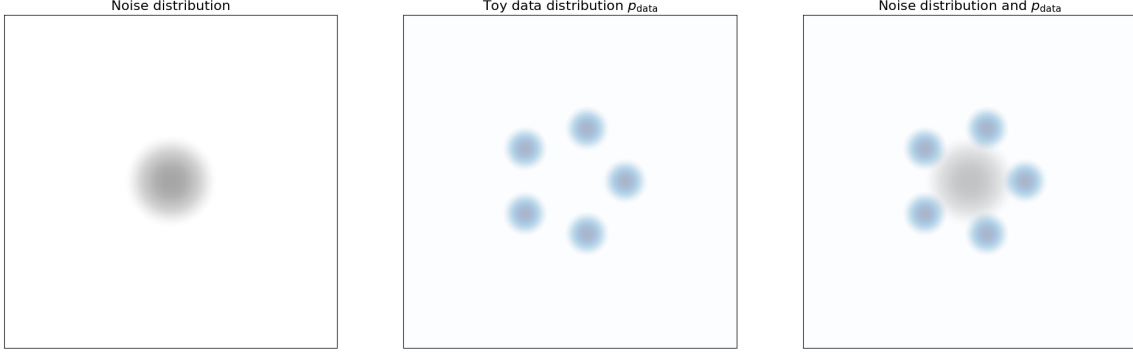


Figure 7: Noise and toy Gaussian data distributions

Next, simulated the corruption process on 1000 samples from p_{data} for $t \in \{0, 0.25, 0.5, 0.75, 1\}$. The results of the corruption progress is shown in Figure 8.

It can be seen from Figure 8 that the samples start out as the centered Gaussian noise distribution at $t = 0$ and then gradually converge towards the toy distribution as the corruption process propagates in time. At $t = 1$, we reach a distribution that exactly matches the one shown in figure (7). From the conditional path defined in Equation 21 along with the linear noise schedulers in Equation 22 and Equation 23 we have:

$$x_t = tz + (1 - t)\epsilon \quad (24)$$

So we should expect:

$$t = 0 \implies x_t = \epsilon, \quad t = 1 \implies x_t = z \quad (25)$$

Meaning our results are completely in line with theory, given the problem definition.

(d) Modify the noise schedules so that they are non-linear (while still satisfying the monotonicity constraints above). Then, generate a plot analogous to panel (c) using the new schedules. How does changing the schedule influence the corruption process?

Answer: We decided to implement quadratic noise schedules, namely:

$$\alpha_t = t^2, \quad (26)$$

$$\beta_t = 1 - t^2. \quad (27)$$

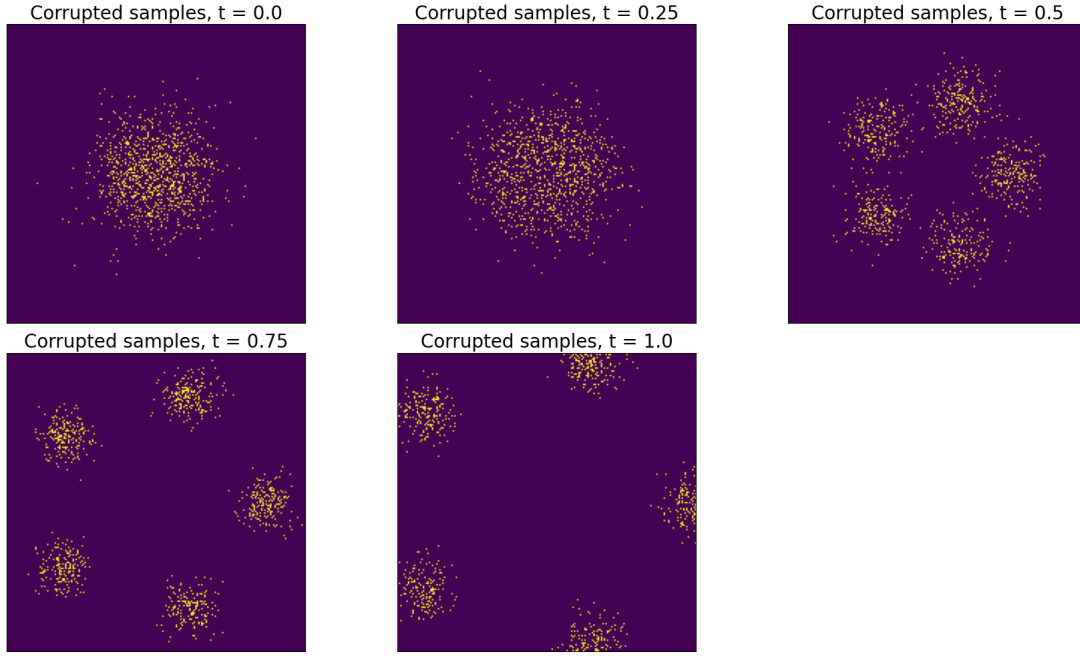


Figure 8: Corruption process for linear α_t and β_t on 1000 samples for $t = 0, 0.25, 0.50, 0.75, 1.0$.

These two functions are both continuously differentiable and satisfy the conditions $\alpha_0 = \beta_1 = 0$ and $\alpha_1 = \beta_0 = 1$. We then simulated the corruption process on 1000 samples for $t = 0, 0.25, 0.50, 0.75, 1.0$, and the quadratic noise schedules. The results are shown in Figure 9 and it is clear that the quadratic noise schedule results in a slower corruption process for $t < 1$, as compared to the linear noise schedule. As t approaches 1 the corruption accelerates rapidly leading the same corruption distribution for both the quadratic and linear schedules at $t = 1$.

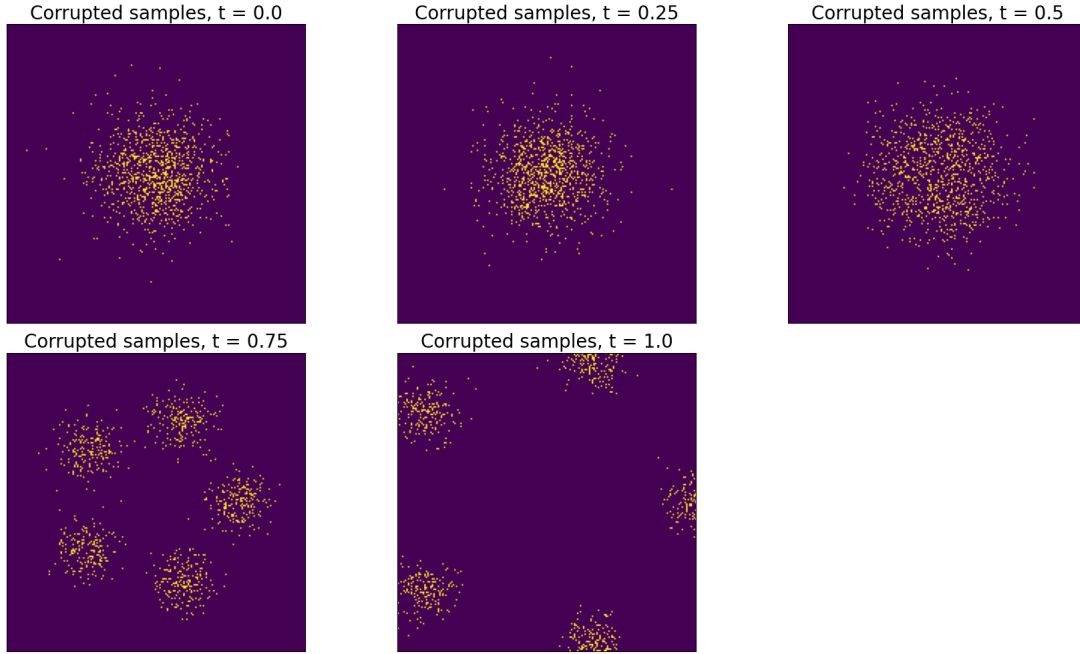


Figure 9: Corruption process for quadratic α_t and β_t on 1000 samples for $t = 0, 0.25, 0.50, 0.75, 1.0$.

3. There are many ways to train generative models.

In this case, we want to train our generative model by minimizing the conditional flow matching loss given by:

$$L(\theta) = E_{t,z,x} \left[|u_t^\theta(x) - u_t(x|z)|^2 \right] \quad (28)$$

Here, $t \sim U[0, 1]$, $z \sim p_{data}$ and $x \sim p_t(\cdot|z) = \mathcal{N}(\alpha_t z, \beta_t^2 I)$, same as x_t before.

(a) The conditional vector field is given by:

$$u_t(x|z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x \quad (29)$$

Find a simplified expression for $u_t(z_t | z)$ when x is drawn from the conditional path $p(\cdot|z)$, and implement it.

Answer: In the case where x_t is drawn from the conditional path then we have $x = x_t = \alpha_t z + \beta_t \epsilon$,

and we can plug this into Equation 29 which yields:

$$\begin{aligned}
u_t(x_t|z) &= \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} (\alpha_t z + \beta_t \epsilon) \\
u_t(x_t|z) &= \dot{\alpha}_t z - \frac{\dot{\beta}_t}{\beta_t} \alpha_t z + \frac{\dot{\beta}_t}{\beta_t} \alpha_t z + \frac{\dot{\beta}_t}{\beta_t} \beta_t \epsilon \\
&\therefore \boxed{u_t(x_t|z) = \dot{\alpha}_t z + \dot{\beta}_t \epsilon}.
\end{aligned} \tag{30}$$

(b) Implement an MLP architecture that takes $(x, t) \in \mathbb{R}^3$ as inputs and outputs the estimated vector field $u_\theta^t(x_t) \in \mathbb{R}^2$.

Answer: We implemented an MLP architecture that takes (x, t) as inputs and outputs the estimated vector field $u_\theta^t(x_t) \in \mathbb{R}^2$ in our `.ipynb` notebook. We implemented the MLP such that the number of hidden layers could be defined by the user, as well as the dimensions of the input data. We used the `torch.nn.SiLU` activation function.

(c) Implement a training loop according to Algorithm 1, and train an MLP with 4 hidden layers of dimension 64 on the 2D toy dataset. Choose noise schedulers α_t, β_t according to your liking (that fulfill the needed criteria), and state what you used. Does the loss converge?

Answer: Next, we implemented a training loop according to Algorithm 1 shown below.

Algorithm 1: Conditional flow matching

Require: data set with samples p_{data} , vector field u_t^θ

for each batch do

- Sample data point $z \sim p_{\text{data}}$.
- Sample random time $t \sim U[0, 1]$.
- Sample noise $\epsilon \sim \mathcal{N}(0, I)$.
- Set $x_t = \alpha_t z + \beta_t \epsilon$.
- Compute loss $L(\theta) = \|u_t^\theta(z_t | z) - u_t(x_t | z)\|^2$.
- Update model parameters θ using gradient step on $L(\theta)$.

end

For our training loop we chose **linear** noise schedulers for α_t and β_t as we felt they were the simplest option and a good starting point. We ran the training loop for 1000 epochs, plotting the loss function after each epoch. The results are shown in Fig 10 and it is clear that the loss converges, after approximately 600 epochs (while still oscillating around the same loss value).

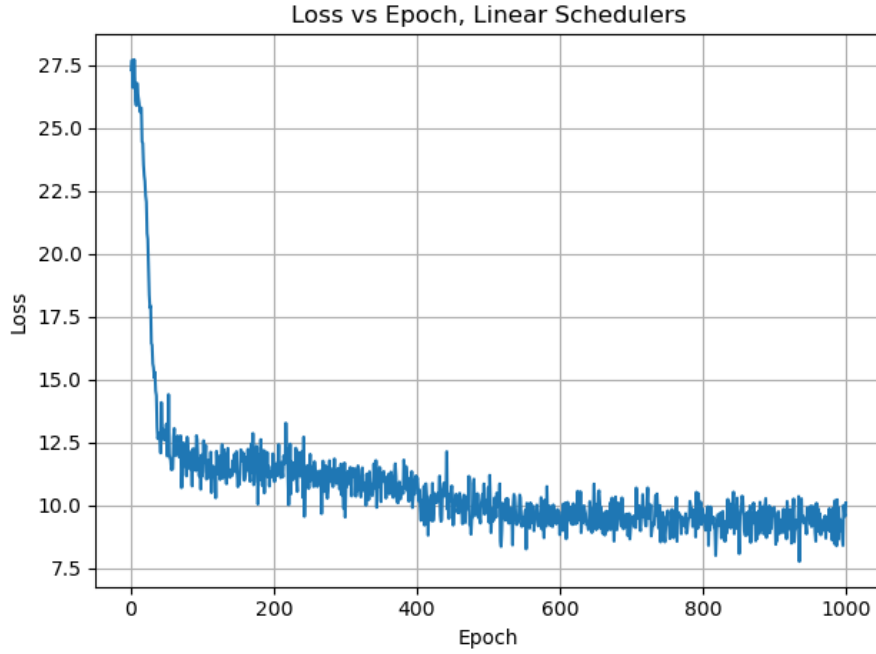


Figure 10: Loss as a function of epoch for linear noise schedulers α_t and β_t .

(d) Let the number of time steps $n = 1000$, and sample 300 realizations from p_{data} . Using the Euler method ($a_t = 0$) applied to the ODE parametrized by your trained vector field, plot a 2D scatterplot of 300 corrupted data points according to the noise scheduler for $t = 0, 0.25, 0.50, 0.75, 1$. Finally, simulate the full path for the 300 sampled data points together with a plot of the simulated trajectories X_t . How does the choice of n affect the sampling?

Answer: We began by testing our helper class, `ReverseProcess`, and used it to create 1000 samples of our Gaussian mixture p_{data} , and then in turn used this to create 1000 samples of the function $x_t = \alpha_t z + \beta_t \epsilon$. We did this for $t = 0.0, 0.25, 0.5, 0.75, 1.0$ and for the linear noise schedulers. The results are shown in Figure 11 and we can clearly see the points spreading out from the IsotropicGaussian (grey blob) to converge on the GaussianMixture (blue blobs) over time.

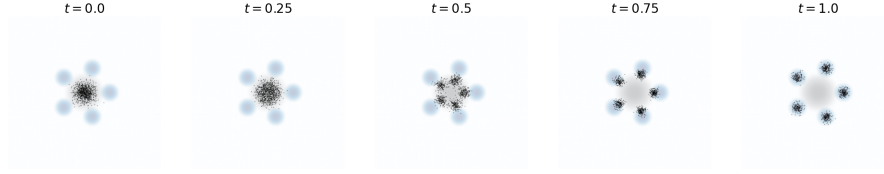


Figure 11: Behavior of x_t for 1000 samples and varying values of t .

Now that we had our helper class working, we sampled 300 realizations from p_{data} and compared the trajectories of the corrupted data points to the trajectories learned from our SDE. We did this for time step values of $n_t = 5, 10, 100, 1000$ and for each time step we showed the positions of the points at (approximately) $t = 0, 0.25, 0.50, 0.75, 1$. Based on the `record_every()` function provided, the values of t were not exactly equal to 0, 0.25, 0.5, 0.75, 1 for each value of the timesteps.

The results for each timestep step values are shown in Figure 12 to Figure 15 and it is clear that the learned SDE fits to the data better with a finer resolution, ie a larger number of timesteps. The best fit is achieved in Figure 15 with $n_t = 1000$ timesteps.

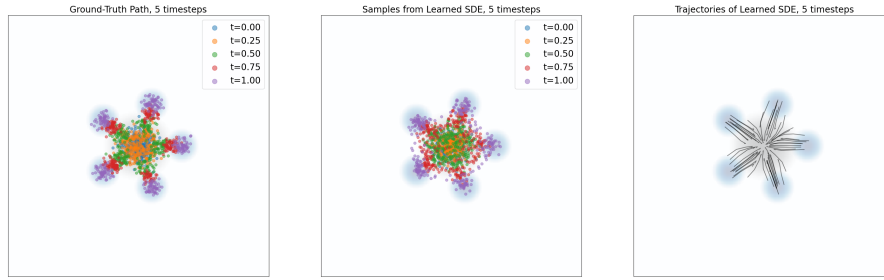


Figure 12: The true position of the 300 sampled data points for varying values of t (left), the position of the data points learned from our SDE (center), and the trajectories learned from our SDE (right). Each plot is generated for 5 timesteps between $t = 0$ and $t = 1$.

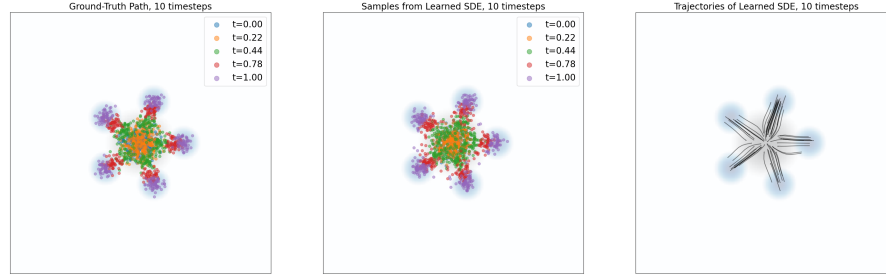


Figure 13: The true position of the 300 sampled data points for varying values of t (left), the position of the data points learned from our SDE (center), and the trajectories learned from our SDE (right). Each plot is generated for 10 timesteps between $t = 0$ and $t = 1$.

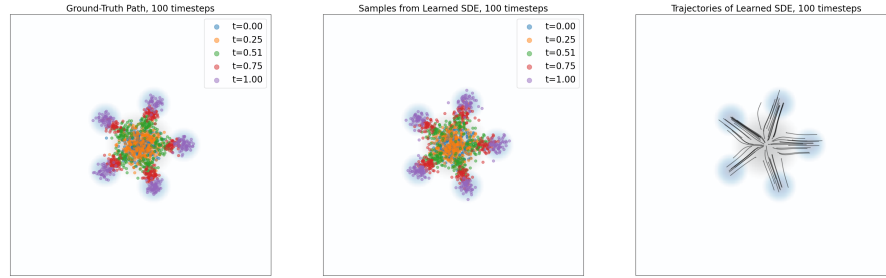


Figure 14: The true position of the 300 sampled data points for varying values of t (left), the position of the data points learned from our SDE (center), and the trajectories learned from our SDE (right). Each plot is generated for 100 timesteps between $t = 0$ and $t = 1$.

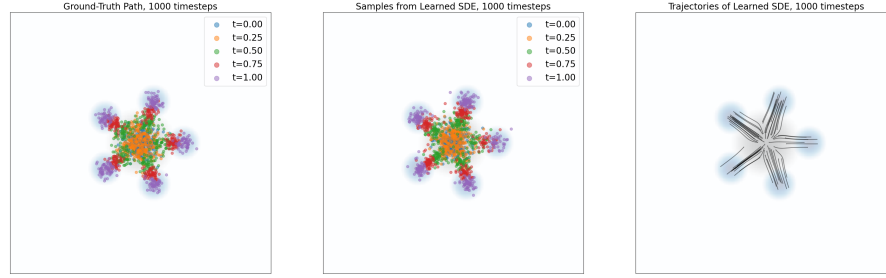


Figure 15: The true position of the 300 sampled data points for varying values of t (left), the position of the data points learned from our SDE (center), and the trajectories learned from our SDE (right). Each plot is generated for 1000 timesteps between $t = 0$ and $t = 1$.

4. Let us now turn to image generation.

To handle high-dimensional image data, we need another architecture than an MLP to parameterize our vector field. We will use the famous UNet architecture with some modification to allow for time embedding (the network must be informed about the current time t and for CNNs it is not as straightforward as for MLPs where we just feed it as an additional input).

(a) Select a few data points from your choice of image data set, and provide a plot of the corresponding corrupted data points according to $t = 0, 0.25, 0.50, 0.75, 1$.

Answer: We selected 9 data points from the brain-stroke data set and sampled the dataset using our `ReverseProcess.sample_dataset()` function. Each image had 32×32 pixels (and 1 channel) which resulted in $32 \times 32 = 1024$ data points, starting as Gaussian noise for each image sample at $t = 0.0$. We then corrupted the data points using the noise schedulers and show this corruption process for $t = 0, 0.25, 0.5, 0.75, 1$ in Figure 16. We can see the data points in a Gaussian distribution at $t = 0.0$ and ending in the position of real brain-stroke images at $t = 1.0$.

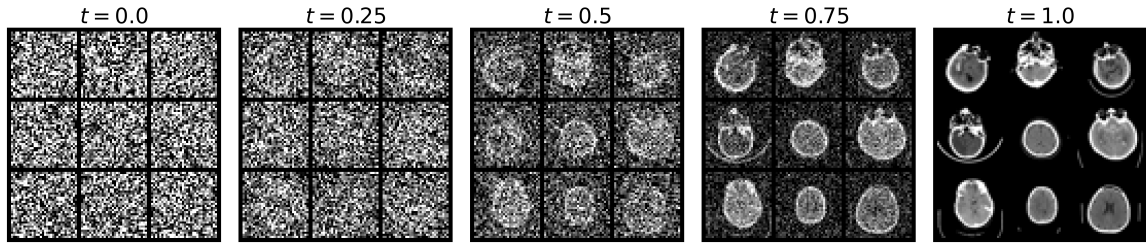


Figure 16: The progression of the corrupted data points from $t = 0.0$ to $t = 1.0$ with linear noise schedulers for the brain-stroke dataset.

(b) Implement and record a smaller layer of the UNet architecture and add the pre-defined time embedding according to Figure 17. Train a UNet over 5000 epochs on the image data according to step (a) using your UNet.

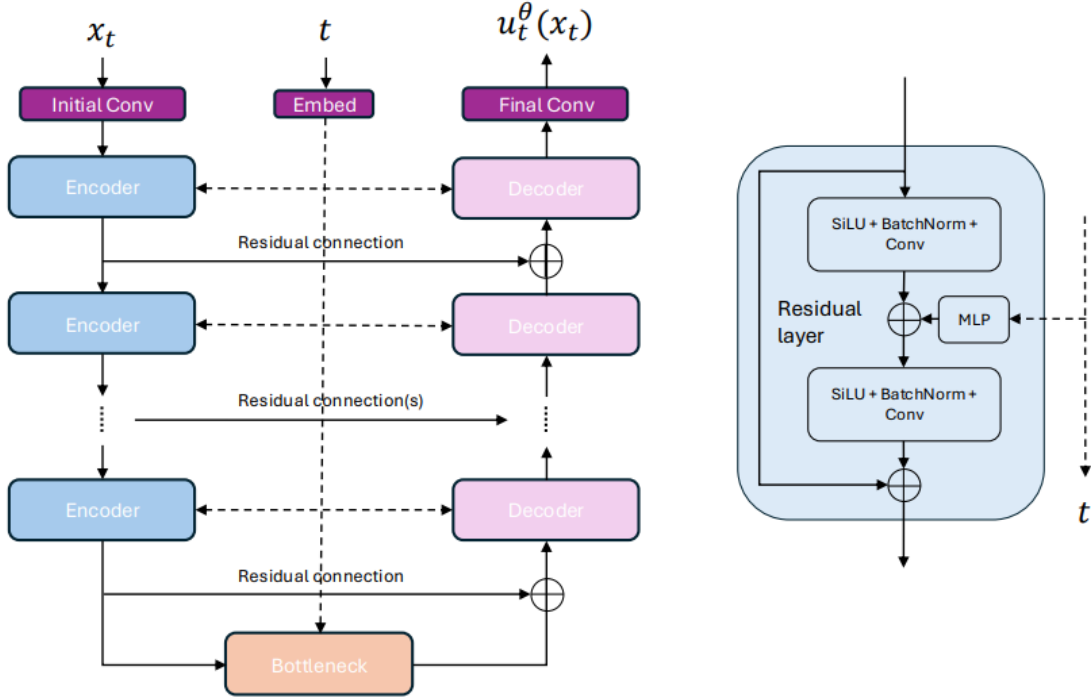


Figure 17: UNet architecture together with a detailed overview of the residual layer. Each encoder consists of two residual layers followed by downsampling. Each decoder consists of upsampling followed by two residual layers. The bottleneck module consists of three residual layers.

Answer: We implemented a UNet architecture according to Figure 17, and used the pre-defined time embedding that is shown in the figure and provided in the `.ipynb` notebook. We build a model that consisted of a first block (block 1), followed by a time adapter, followed by a second block (block 2). Both block 1 and block 2 are comprised of three layers, and the layers are as follow. First, a convolution is applied, then the data is normalized, and finally we apply the SiLU activation function.

The UNet architecture is also such that before applying our layers we take a copy of the data. We then apply block 1, followed by the time adapter, followed by block 2, to our data before adding back the copied residual data to our output.

Once our model was defined, we then trained the UNet over 5000 epochs on the brain-stroke image data and the linear noise schedulers.

(c) Sample a couple of images from the image distribution by solving the flow ODE

$$dX_t = u_\theta^t(X_t) dt$$

using your trained UNet as the drift term.

Answer: We sampled initial noise from our initial noise distribution of $\epsilon \sim \mathcal{N}(0, I)$ 9 times. Each initial noise sample contained 1 channel of 32×32 pixels. The 9 initial noise distributions are shown in Figure 18.

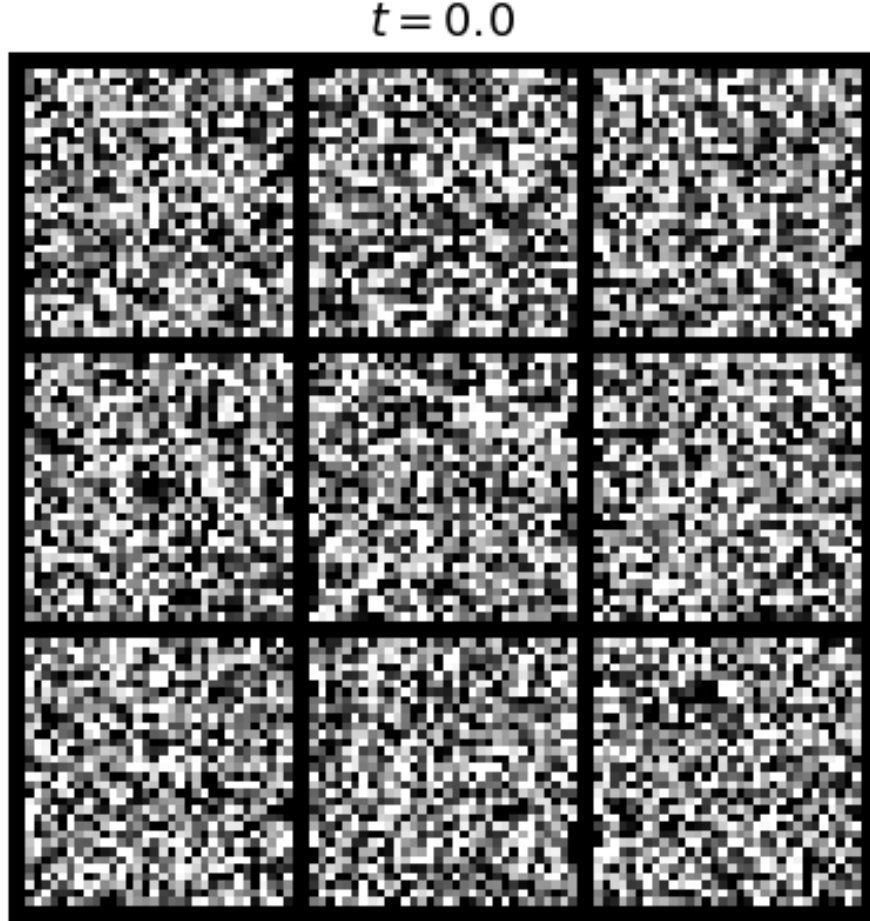


Figure 18: The initial noise distributions are shown, sampled from our initial noise distribution of $\epsilon \sim \mathcal{N}(0, I)$.

We then used the learned ODE from our UNet model and the `EulerMaruyamaMethod` to simulate 9 images from our dataset, shown in Figure 19. The images look remarkably like those from the dataset and we were very impressed with these results.

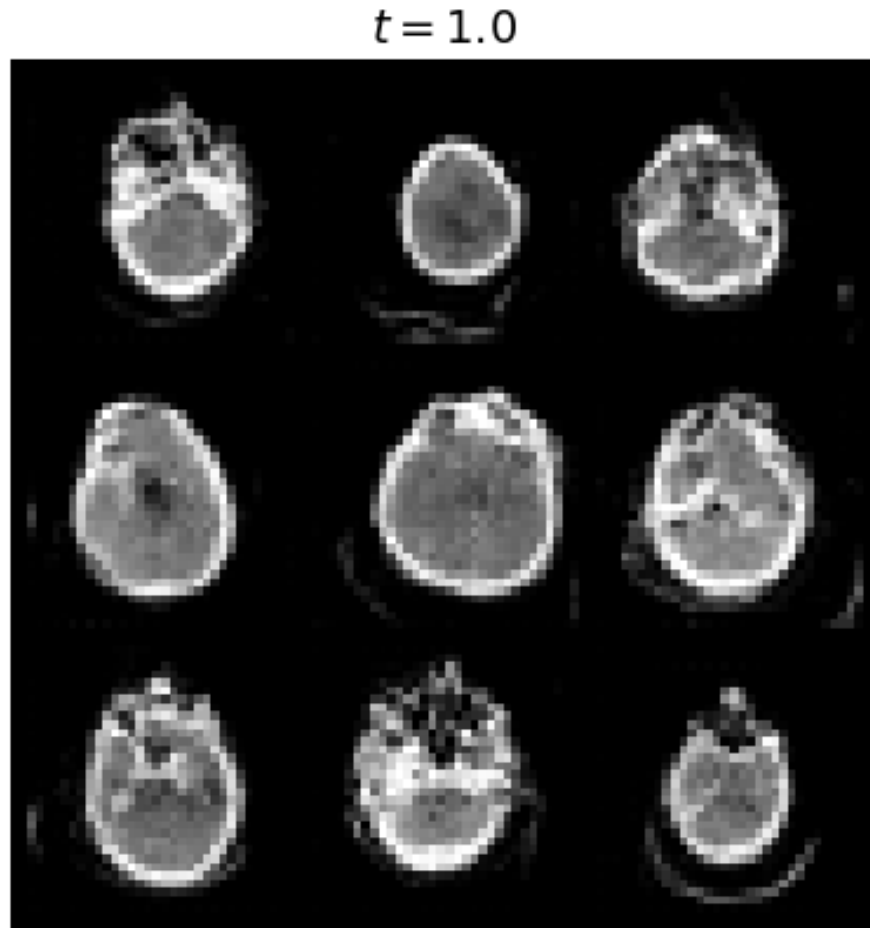


Figure 19: The 9 generated images are shown, for the UNet model trained with the linear noise schedulers.

(d) Play around with different noise schedulers and comment on how they affect the sample quality.

Answer: We tested three different noise schedulers, the linear noise scheduler (with results presented in the previous section), the quadratic noise scheduler (defined according to Equation 26 and Equation 27), and a sine/cosine noise scheduler. We implemented the sine/cosine noise schedulers as

$$\alpha_t = \sin\left(\frac{\pi t}{2}\right), \quad (31)$$

$$\beta_t = \cos\left(\frac{\pi t}{2}\right). \quad (32)$$

These two functions are both continuously differentiable and satisfy the conditions $\alpha_0 = \beta_1 = 0$ and $\alpha_1 = \beta_0 = 1$.

Now, let's present our results from the quadratic noise schedulers. We present these results in Figure 20 and it is clear that the model was also a success.

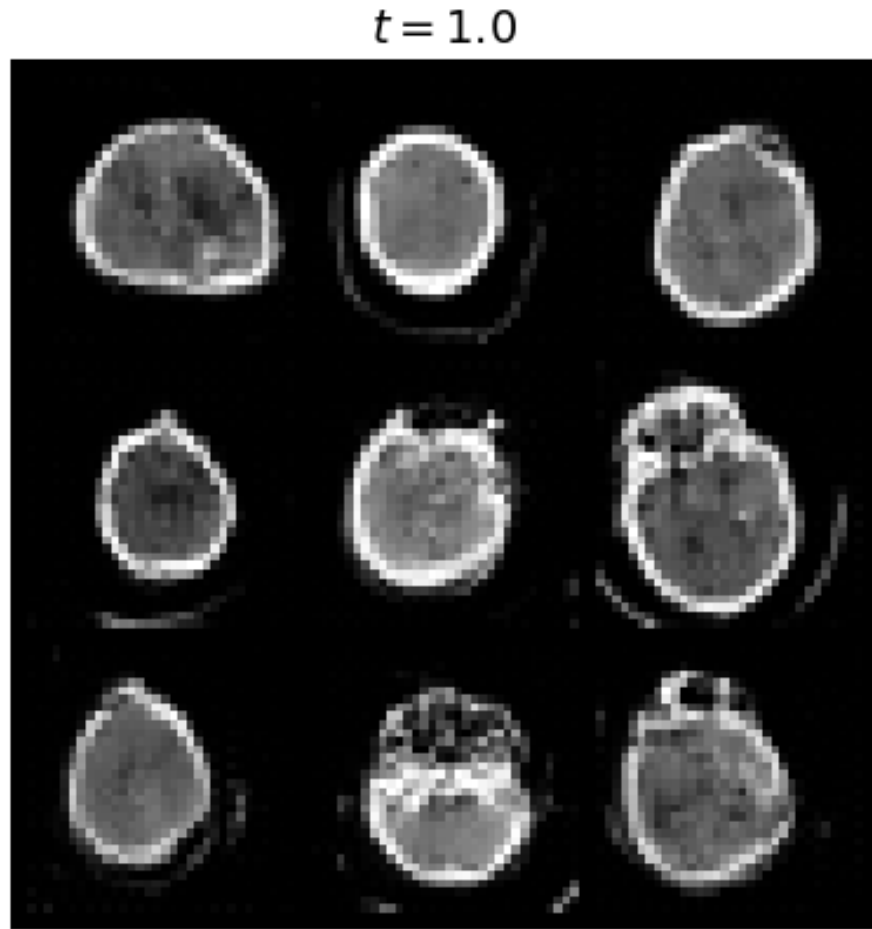


Figure 20: The 9 generated images are shown, for the UNet model trained with the quadratic noise schedulers.

Finally, we present our results from the sine/cosine schedulers in Figure 21. These schedulers worked very well and we can again see nice image data.

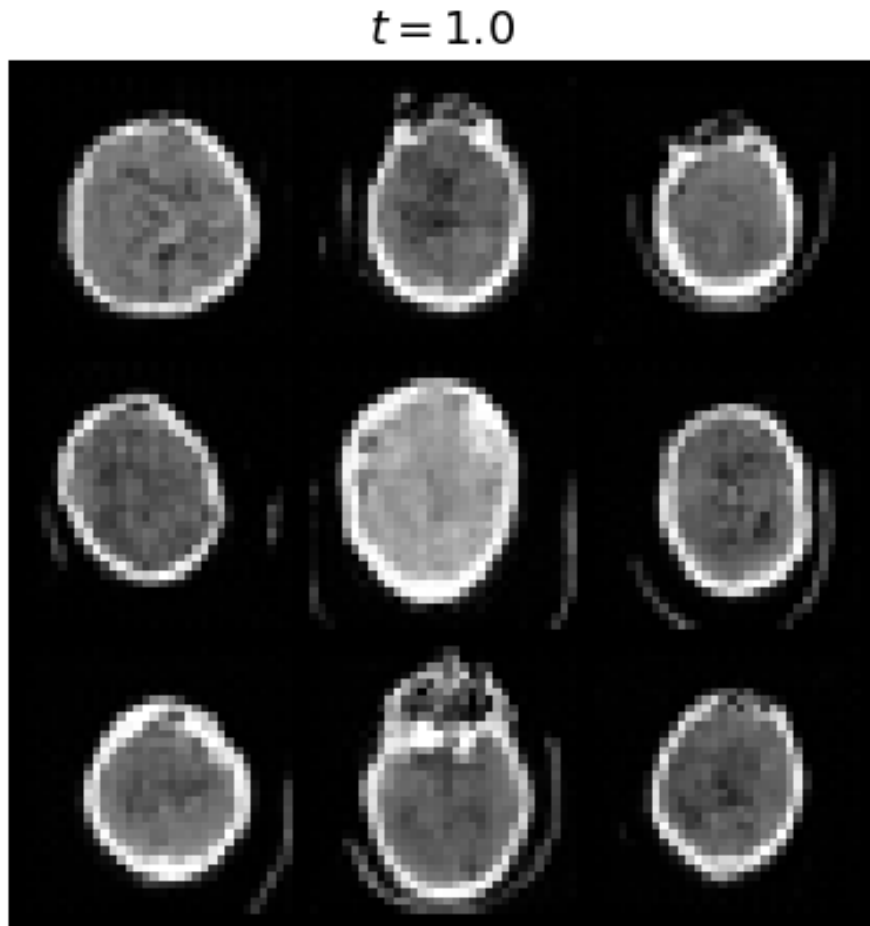


Figure 21: The 9 generated images are shown, for the UNet model trained with the quadratic sine/cosine schedulers.

Now we can compare the sample quality between noise schedulers. The linear schedulers (Figure 19), the quadratic schedulers (Figure 20), and the sine/cosine schedulers (Figure 21). The sample quality between all three noise schedulers is very comparable, although the sine/cosine schedulers lead to smoother and less pixelated image quality, as compared to the linear and quadratic schedulers. Additionally, I feel the quadratic schedulers also lead to less pixelated data than the linear schedulers, although it is a very slight difference.

5. So far we have only trained flow models (no diffusion term).

Via the Fokker–Planck equation, it can be shown that the following SDE

$$dX_t = \left(u_\theta^t(X_t) + \frac{\sigma^2}{2} \nabla \log p_t(X_t) \right) dt + \sigma dW_t, \quad (33)$$

has the same probability paths p_t as the flow ODE

$$dX_t = u_\theta^t(X_t) dt.$$

Hence, to convert the flow model into a diffusion model, we have to learn the second drift term $\nabla \log p_t(X_t)$, called the *score function*. This can be done by training a network u_θ^t using conditional score matching:

$$L(\theta) = \mathbb{E}_{t \sim U[0,1], z \sim p_{\text{data}}, z_t \sim p(\cdot|z)} \|u_\theta^t(z_t) - \nabla \log p(z_t | z)\|^2.$$

(a) Derive an explicit formula for the conditional score function $\nabla \log p_t(x | z)$ from the conditional path and implement it. Then train a score network s_θ^t on the image dataset using the conditional score matching loss.

Answer:

We know that $p_t(\cdot|z) = \mathcal{N}(\alpha_t z, \beta_t^2 I)$; the d-dimensional Gaussian probability density function is:

$$p(x) = (2\pi)^{-d/2} \det(\Sigma)^{-1/2} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right) \quad (34)$$

In our case, Σ is diagonal with $\Sigma(i, j) = \beta_t^2, \forall i = j$ and $\Sigma(i, j) = 0, \forall i \neq j$:
 $\implies \det(\Sigma)^{-1/2} = (\beta_t^{2d})^{-1/2}$. The exponential term can be expressed as:

$$(x - \mu)^T \Sigma^{-1} (x - \mu) = \sum_{i=0}^d \frac{(x_i - \mu_i)^2}{\Sigma(i, i)}$$

But since $\Sigma(i, i) = \beta_t^2, \forall i$:

$$(x - \mu)^T \Sigma^{-1} (x - \mu) = \frac{\|x - \mu\|^2}{\beta_t^2}$$

Using the isotropic properties of Σ along with $\mu = \alpha_t z$, equation (34) reduces to:

$$p(x|z) = \frac{1}{(2\pi\beta_t^2)^{d/2}} \exp \left(-\frac{\|x - \alpha_t z\|^2}{2\beta_t^2} \right) \quad (35)$$

Now, we take the logarithm of eq (35):

$$\begin{aligned} \log p_t(x|z) &= \log \left(\frac{1}{(2\pi\beta_t^2)^{d/2}} \exp \left(-\frac{\|x - \alpha_t z\|^2}{2\beta_t^2} \right) \right) \\ \implies \log p_t(x|z) &= \log \left(\frac{1}{(2\pi\beta_t^2)^{d/2}} \right) + \log \left(\exp \left(-\frac{\|x - \alpha_t z\|^2}{2\beta_t^2} \right) \right) \\ \implies \log p_t(x|z) &= \log \left(\frac{1}{(2\pi\beta_t^2)^{d/2}} \right) - \frac{\|x - \alpha_t z\|^2}{2\beta_t^2} \end{aligned}$$

The gradient with respect to x gives:

$$\nabla_x \log p(x|z) = -\frac{2(x - \alpha_t z)}{2\beta_t^2} = \frac{\alpha_t z - x}{\beta_t^2} \quad (36)$$

(b) Sample new data using the diffusion model by combining the trained vector field u_θ^t and the score network s_θ^t and simulate the SDE in (3). How does the value of σ affect the samples?

Answer:

Final question: Did you use an AI tool (other than the machine learning models you trained in this exercise) for anything else than information searching when solving this problem set? If so, please write a **brief statement of how you used AI**.

Answer: Here is my AI usage for each task.

- Task 0: We used Chat-GPT to generate the latex code for the provided assignment PDF. We then edited this latex code and filled in my own answers beneath each task.

- Task 1: Scott - None.

Gustav -

- Task 2: Scott - I used Chat-GPT to help me generate the code for making subplots, to get the nice plots in Figure 8 and Figure 9. I also asked Chat-GPT about what type of non-linear noise schedulers I could implement and was inspired by the response of quadratic that it gave me.

Gustav -

- Task 3: Scott - I used Chat-GPT to help me rewrite the `record_every()` function so that the number of timesteps would always be partitioned into 5 (to better approximate $t = 0, 0.25, 0.5, 0.75, 1$ for all timestep values). I was unsatisfied that the function was generating 6 t values with $n_t = 10$ timesteps.

Gustav -

- Task 4: Scott - I used Chat-GPT for help with deciding on a third noise scheduler to use, and implemented the cosine/sine noise scheduler at his suggestion.

Gustav -

- Task 5: Scott - None.

Gustav -

Total number of points: 25

Motivate your answers wherever applicable.

Good luck!